



中国科学院大学

University of Chinese Academy of Sciences

研究生学位论文中期报告

报告题目 面向深度学习框架应用接口的模糊测试优化方法研究

学生姓名 潘哲 学号 202318018670142

指导教师 史岗 职称 正高级工程师

学位类别 工学博士

学科专业 网络空间安全

研究方向 系统安全理论与技术

研究所（院系） 中国科学院信息工程研究所

填表日期 2026 年 09 月 26 日

中国科学院大学制

目 录

一、选题的背景及意义、发展现状和趋势	1
1.1 选题背景及意义.....	1
1.2 发展现状和趋势.....	4
二、课题主要研究内容、预期目标	11
2.1 研究内容.....	11
2.2 预期目标.....	12
三、学位论文进展情况、已取得阶段性成果	12
3.1 基于代码演化特征的 API 风险评估与测试资源分配方法.....	12
3.2 面向复杂约束的 API 输入生成增强方法	15
3.3 参数感知的 API 输入突变方法	17
四、下一步工作计划和内容、预计答辩时间及存在的问题	20
4.1 下一步工作计划.....	20
4.2 预计答辩时间.....	20
4.3 存在的问题与挑战.....	21
五、已取得研究成果列表（已发表、待发表学术论文等）	21
5.1 学术论文.....	21
5.2 科研项目参与.....	21
5.3 缺陷发现与修复.....	21
参考文献.....	23

一、选题的背景及意义、发展现状和趋势

1.1 选题背景及意义

在信息化与智能化快速发展的背景下，海量数据的生成与积累为人工智能技术的发展提供了重要基础。2006 年，Hinton 等人的研究工作推动了深层神经网络训练方法的发展，使深度学习（Deep Learning，DL）进入快速发展阶段^[1]。与传统机器学习方法相比，深度学习通过多层神经网络结构展现出更强的特征表示能力，能够从海量高维数据中自动提取复杂模式。随着图形处理器（Graphics Processing Unit，GPU）、张量处理器（Tensor Processing Unit，TPU）等高性能并行计算设备的广泛应用，以及数据存储成本的显著下降，深度学习的应用潜力得以进一步释放^[2]。在这一过程中，如何高效构建、训练和部署深度学习模型逐渐成为亟待解决的关键问题。为降低模型开发复杂度、提高计算资源利用效率，深度学习框架应运而生。

在这一背景下，一系列功能强大、生态完善的深度学习框架相继出现。例如，Google 推出的 TensorFlow^[3]早期以静态计算图机制为代表，适合大规模分布式训练；Facebook 开发的 PyTorch^[4]采用动态图计算机制，强调灵活性和易用性；华为推出的 MindSpore 则面向端、边、云等多种场景，支持不同硬件环境下的统一开发与高效训练。此外，MXNet^[5]、CNTK^[6] 等框架也在不同应用场景中发挥了重要作用。这些框架通过封装自动求导、张量计算和硬件调用等底层功能，为深度学习模型的开发、训练和部署提供了统一的软件基础。

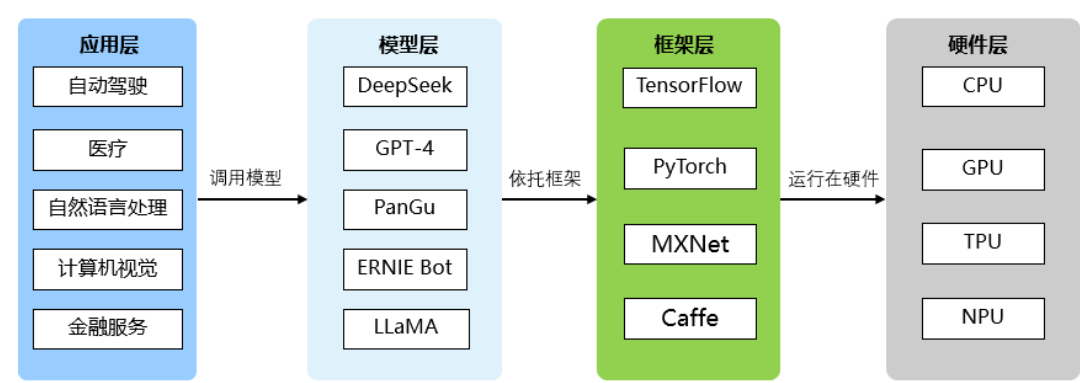


图 1-1 深度学习框架应用示意图

随着人工智能在各行业中的广泛应用，深度学习框架的基础支撑作用进一步凸显。如图 1-1 所示，框架向上支撑算法与应用开发，广泛服务于医疗^[7,8,9,10]、自动驾驶^[11,12,13,14]和金融服务^[15,16,17,18] 等对可靠性和安全性要求较高的领域；向下则连接 GPU、TPU、NPU 等异构计算平台，为模型提供高效的计算与运行环

境。因此，深度学习框架已经成为连接上层人工智能应用与底层计算资源的重要基础设施软件。

以大模型为例，2023 年华为发布的 PanGu- Σ ^[19] 大语言模型，其参数规模高达 1.085 万亿，其大规模训练依赖 MindSpore 在并行计算、内存优化与分布式训练等方面提供的框架支持。随着模型规模和计算复杂度持续增长，深度学习框架在资源调度、计算优化和模型运行中的作用日益重要。因此，深度学习框架常被视为人工智能领域的“操作系统”，成为支撑人工智能研究与应用规模化发展的重要软件基础。

尽管深度学习框架为人工智能模型的开发、训练和部署提供了重要支撑，但随着框架功能不断扩展，其代码规模、软件依赖和接口数量也持续增长，系统复杂性随之提高。一方面，深度学习框架通常依赖数值计算库、并行计算接口和系统工具库等大量第三方组件，任一环节存在缺陷都可能影响框架整体的稳定性与安全性；另一方面，框架自身的实现缺陷也可能向上传递至模型训练、推理和实际应用过程。在自动驾驶等高可靠性应用场景中，深度学习系统的异常行为可能造成严重后果，例如 2016 年谷歌自动驾驶汽车曾因系统对道路环境判断不当发生交通事故^[20]。此外，基于 PyTorch 构建的开源机器翻译框架 Sockeye 在 2.3.24 之前的版本中曾因不安全的 YAML 文件加载方式存在代码注入风险^[21]。这些案例进一步表明，深度学习相关基础软件的可靠性与安全性对上层人工智能应用具有重要影响。

从公开漏洞数据来看，深度学习框架自身同样面临较为突出的安全风险。以 TensorFlow 为例，如图 1-2 所示，迄今已公开披露的漏洞数量累计超过 430 个，其中大量漏洞具有较高的 CVSS 危险性评分。随着深度学习框架持续演化以及软硬件支持范围不断扩大，其内部实现和接口行为日趋复杂，潜在缺陷可能导致程序崩溃、异常计算、服务中断等问题。因此，如何通过自动化手段及时发现深度学习框架中的潜在缺陷，已成为保障人工智能基础软件可靠性与安全性的重大问题。

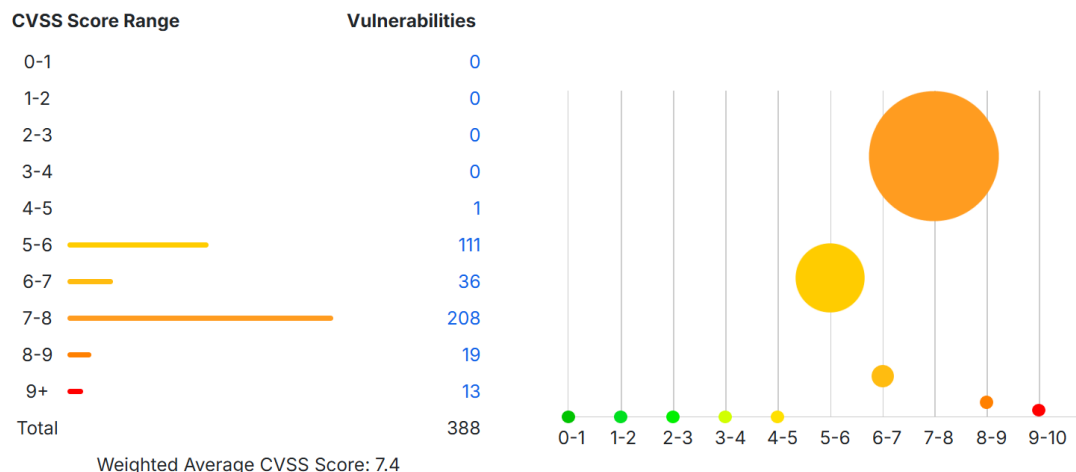


图 1-2 TensorFlow 框架已公开漏洞的 CVSS 危险性分布

面对深度学习框架庞大的接口规模和复杂的输入空间，传统测试方法在测试充分性方面存在一定局限。人工测试和单元测试通常依赖预先设计的测试用例，难以系统探索张量形状、数据类型、索引、属性以及不同参数组合形成的大规模输入空间；静态分析虽然能够在不执行程序的情况下发现部分潜在问题，但对于运行时张量形状、跨层调用以及复杂参数依赖关系的精确刻画仍较为困难。相比之下，模糊测试能够自动生成或变异大量测试输入，并通过真实执行发现崩溃、异常和计算不一致等行为，从而降低人工测试用例设计成本，并扩大对复杂输入空间和边界条件的探索。因此，模糊测试逐渐成为深度学习框架自动化缺陷检测的重要技术手段。

根据测试对象的不同，面向深度学习框架的模糊测试主要可以分为模型级模糊测试（Model-level Fuzzing）和 API 级模糊测试（API-level Fuzzing）^[22]。模型级模糊测试通常以完整的深度神经网络模型及其输入作为测试载体，通过模型生成或突变并结合差分测试发现框架执行过程中的潜在缺陷^[23]。这类方法能够从较完整的模型执行场景验证框架行为，但模型构造与执行开销相对较大，且算子之间的依赖关系限制了对单个 API 特殊参数组合和边界条件的针对性探索。相比之下，API 级模糊测试直接面向框架接口构造和变异输入，具有测试粒度细、便于大规模执行以及缺陷定位相对直接等特点，因此逐渐成为深度学习框架测试的重要研究方向，也是本文重点关注的测试层次。

然而，深度学习框架 API 数量众多、输入约束复杂且参数之间存在密切依赖，在有限测试资源下实现充分而高效的输入空间探索仍面临较大挑战。因此，有必要围绕 API 级模糊测试的关键环节开展更加有针对性的优化研究，以提高测试资源利用效率、输入探索能力和潜在缺陷触发能力。

开展面向深度学习框架应用接口的模糊测试优化方法研究，具有重要的研究意义与应用价值。首先，有助于提升深度学习库模糊测试的有效性与测试资源利

用效率。针对深度学习框架 API 数量众多、输入约束复杂以及参数依赖密集等特点,通过研究测试预算分配、有效输入生成和参数感知突变等方法,可以在有限测试资源下提高有效输入获取能力与执行分支探索水平,从而增强复杂 API 和潜在缺陷的自动化探索能力。其次,有助于为深度学习基础软件的安全性及可靠性保障提供技术支撑。通过发现并推动修复框架底层实现中的潜在缺陷,可以降低模型训练与推理所依赖的软件基础中存在的运行风险,为深度学习框架的版本迭代、缺陷修复和安全维护提供支持。最后,本研究在测试资源分配、复杂输入构造以及参数组合探索等方面形成的方法与经验,也可为其他具有接口数量多、输入约束复杂和参数依赖关系密集等特征的软件库开展自动化测试提供一定的参考。

1.2 发展现状和趋势

近年来,围绕深度学习框架可靠性与安全性问题,国内外研究者开展了大量自动化测试研究。其中,模糊测试因能够面向复杂输入空间持续生成和变异测试用例,已经形成较为系统的研究体系。根据测试对象的粒度不同,现有研究主要包括模型级模糊测试和 API 级模糊测试两类。模型级方法侧重从完整神经网络模型的执行行为验证框架实现,API 级方法则进一步面向单个接口或算子开展细粒度测试。随着研究深入,相关工作逐渐由模型整体行为验证向 API 级输入生成、约束推理、覆盖探索和参数突变等方向发展。

1.2.1 面向深度学习框架的模型级模糊测试

模型级模糊测试以完整的深度神经网络模型及其输入为测试对象,通过模型生成或突变构造测试用例,并利用跨框架或跨后端的执行差异发现潜在缺陷。其基本流程包括模型构造、权重与输入生成、执行及差分检测,并根据反馈继续扩展测试样例^[24]。

早期研究主要围绕已有模型的差分验证与突变展开。CRADLE^[25]比较同一模型在不同后端的输出及隐藏层行为,用于发现和定位框架缺陷,但对已有模型的依赖限制了探索范围。AUDEE^[26]引入基于遗传算法的搜索,联合改变模型结构、权重和输入,以扩大测试空间。LEMON^[27]进一步设计层间与层内突变规则,并结合种子选择和突变调度提升模型多样性。Ramos^[28]则关注算子调用关系,通过调用边突变构造不同算子组合,并利用跨框架 API 映射支持差分测试。

后续工作从结构覆盖与测试效率等方面改进模型探索。FAME^[29]扩展模型突变规则,增加 API 级突变方式;DeepCov^[30]利用模型层间连接与层边覆盖指导测试生成;COMET^[31]结合模型压缩、多维度突变和覆盖度量改进测试效率与充分性;MMOS^[32]采用多阶段突变算子选择与能量分配机制,提高突变调度效率。

此外，NeuRI^[33]和 Prolog^[34]探索约束感知的模型生成，分别利用程序综合和逻辑推理构造满足输入要求的模型。

总体来看，模型级模糊测试已由已有模型的差分验证发展到多样化突变、结构覆盖引导和约束感知生成，但模型构造与执行开销较大，数值误差也会增加缺陷确认难度。同时，模型中的算子依赖限制了对单个 API 复杂参数组合及边界条件的针对性探索，因此仍需结合更细粒度的 API 级测试。

1.2.2 面向深度学习框架的 API 级模糊测试

API 级模糊测试以深度学习框架中的单个 API 或算子为测试目标，通过自动生成或变异调用参数并执行目标接口，检测崩溃、异常、数值不一致等潜在缺陷。与模型级方法相比，API 级测试粒度更细，更便于针对复杂参数组合和边界行为开展系统探索。典型的 API 级模糊测试流程如图 1-3 所示。测试过程首先从目标框架中选择待测 API，并确定相应测试资源；随后构造满足基本调用要求的初始输入，并通过输入生成或突变持续扩大测试空间；生成的测试用例被传递给目标 API 执行，再结合差分测试、运行状态监测和覆盖反馈等方式识别不一致、崩溃及其他异常行为。典型的 API 级模糊测试过程包括待测 API 选择与测试预算确定、初始输入生成、输入突变以及缺陷检测等环节。现有研究主要集中于初始输入生成、输入突变、约束推理和路径探索等方面，而对不同 API 之间测试预算如何进行差异化分配的关注相对较少。下面重点介绍已有研究较为集中的初始输入生成与输入突变方法。

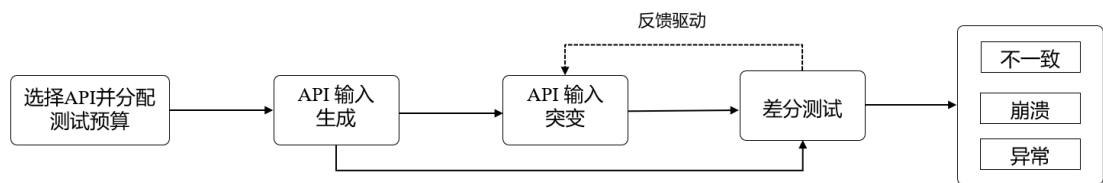


图 1-3 API 级模糊测试基本原理

(1) API 初始输入生成

在 API 初始输入生成领域，研究者提出了多种方法来为目标 API 构造有效的初始输入，为后续测试执行和输入突变提供基础。早期工作主要依赖手工规则或简单的合法输入构造，而近年来，越来越多的研究开始利用文档解析、主动学习、大语言模型和符号执行等技术提取 API 参数约束或生成种子程序，从而提升初始输入的合法性、多样性和缺陷触发潜力。

FreeFuzz^[35]通过运行从开源项目中收集的代码片段来提取 API 的中间过程信息，包括参数类型、取值范围以及输入输出张量的形状，并将其作为种子输入进一步扩展，从而生成覆盖更广的数据样例。考虑到同一 API 在不同硬件平台（如 CPU 和 GPU）上的实现可能存在差异，FreeFuzz 结合差分测试方法

对比不同平台的输出一致性以发现计算错误，同时利用数值精度与运算速度之间的关系开展蜕变测试检测性能缺陷，并通过运行过程监控发现潜在的崩溃与异常。这一方法奠定了深度学习框架 API 级模糊测试的基础。

在 FreeFuzz 的启发下，DeepREL^[36]引入了等价规则来进一步改进差分测试。其核心思想是从 API 功能的角度定义两类等价关系：一是 输出值等价，即在相同输入下，功能相同的 API 应输出一致的结果；二是 输出状态等价，即使输出结果有所不同，功能相似的 API 在执行状态（成功、异常、崩溃）上也应保持一致。DeepREL 采用自然语言处理方法，从 API 文档中识别功能相似或等价的 API 对，并结合 FreeFuzz 生成的输入对其进行差分测试。如果某对 API 的输出不满足等价规则，就可能说明其中存在潜在缺陷。与 FreeFuzz 相比，DeepREL 通过明确化测试预言关系，使得差分测试更具针对性，有助于发现更多类型的缺陷。

除直接生成 API 调用输入外，也有研究在已有输入生成框架的基础上扩展测试目标与测试预言。▽Fuzz^[37]将研究重点转向深度学习框架中的核心组件——自动微分（automatic differentiation）。自动微分是反向传播算法的基础功能，负责屏蔽用户在梯度计算中的复杂细节。▽Fuzz 借鉴 FreeFuzz 的输入生成思路，通过自动创建装饰器函数，将 API 调用抽象为从输入到输出的函数映射，并进一步对每个 API 的函数值及其低阶与高阶梯度进行差分测试。该方法使缺陷检测从普通 API 输入输出扩展到关键训练机制，有助于发现自动微分相关的实现缺陷。

DocTer^[38]提出了一种面向深度学习框架 API 的输入生成方法，其核心思想是利用自然语言处理技术，对官方文档中关于 API 参数的自然语言描述进行依存关系分析，从而自动提取参数的约束条件。具体来说，DocTer 会对文档进行解析，识别参数之间的依赖关系和约束模式，并进一步构建一组可复用的语法规则。这些规则不仅能够帮助研究人员理解 API 参数的取值范围和类型限制，还可以自动生成符合约束的有效输入与边界值输入，避免传统模糊测试中大量无效样例的问题。在此基础上，DocTer 实现了对 API 更加精确和高效的缺陷检测。与传统的随机输入方法相比，该方法能够更大程度地触发潜在的实现漏洞，并减少输入冗余，提高测试覆盖率和缺陷发现率。因此，DocTer 在 API 级模糊测试领域展现出较强的适用性和扩展性，为后续结合文档驱动输入生成研究提供了有益的思路和工具支撑。

与 DocTer 从 API 文档中提取约束不同，ACETest^[39]进一步关注深度学习算子实现中的输入验证逻辑，通过静态分析和符号执行自动识别输入验证路径，并从源码中提取与用户可控参数相关的约束条件。在获得这些约束后，ACETest

利用约束求解生成能够通过合法性检查的测试用例，使输入能够更有效地到达算子核心逻辑。相比依赖人工规则或文档描述的方法，ACETest 减少了对文档完整性和准确性的依赖，能够提取更贴近真实实现的复杂约束，从而提升 API 初始输入生成的有效性。

相比之下，SkipFuzz^[40] 提出了另一条创新路径。不同于依赖人工规则或静态分析的方法，它引入了主动学习（active learning）机制，在模糊测试过程中不断与被测 API 交互，逐步收集和分析 API 的函数特征信息。通过这种动态反馈循环，SkipFuzz 能够在实际测试运行中推断出 API 的输入约束，并据此不断修正和优化输入生成策略。借助这一自适应方式，SkipFuzz 在输入生成的合法性上表现更为出色，能够显著减少无效输入的比例；同时在多样性和覆盖率方面也优于以往研究成果，更容易探索到 API 的深层执行路径。此外，SkipFuzz 的设计有效缓解了传统模糊测试中普遍存在的输入冗余问题，避免了反复触发相同的执行行为而浪费测试资源。实验结果显示，SkipFuzz 不仅能够发现更多触发崩溃的输入，还在多个深度学习框架中验证了其适用性，进一步证明了该方法在提升 API 级模糊测试效率与缺陷发现能力方面的价值。

近年来，大语言模型（LLM）的发展为模糊测试注入了新的活力。TitanFuzz^[41] 利用 Codex 等生成式模型，在给定规范化提示的情况下能够自动生成高质量的种子代码段，从而构建覆盖范围更广、代表性更强的种子库。相比传统依赖人工或启发式方法的方式，这种基于 LLM 的输入生成展现出更强的自动化与泛化能力。进一步地，FuzzGPT^[42] 将历史缺陷数据与 LLM 结合，通过少样本学习和模型微调，使其能够从真实缺陷代码中提炼漏洞特征，并在此基础上生成更具针对性的测试用例。为了验证生成测试用例触发异常的真实性，FuzzGPT 结合差分测试机制，在 CPU 与 GPU 等不同执行环境下对生成代码进行交叉验证，最终检测出 49 个此前未被披露的缺陷。这一系列研究表明，LLM 在生成多样化、边界性和缺陷导向测试输入方面具有较大潜力，有望成为深度学习框架模糊测试的重要辅助技术。

与之类似，FUTURE^[43] 将大语言模型（LLM）与历史漏洞信息深度结合，通过专门化微调，使模型能够自动学习已有缺陷的模式，并生成面向新库的定向测试用例。该方法不仅能够在新兴深度学习库中快速定位潜在漏洞，还能在测试过程中不断积累关于新库 API 的行为特征与异常表现。更为重要的是，这些来自新库的反馈信息会被重新纳入训练与分析流程，用于反向增强现有库的安全性，从而形成一个“由历史到未来、再由未来回溯历史”的安全循环。通过这种循环机制，FUTURE 不仅拓展了 LLM 在软件测试中的应用边界，也为深度学习框架的长期演进提供了动态而持续的安全保障。

PathFinder^[44]提出面向深度学习库的轻量级混合执行方法，根据具体执行样本合成近似路径条件，并通过约束求解生成新的 API 输入。执行结果继续参与条件修正和后续探索，使输入生成能够利用已有路径信息。该方法为路径条件合成与求解相结合的输入生成提供了基础，但近似条件和已有样本未必充分体现复杂 API 的全部输入要求，仍可能出现有效输入获取受阻或覆盖增长停滞。

FlashFuzz^[45]则从覆盖引导模糊测试角度探索 API 级测试。传统覆盖引导模糊器通常以字节流为输入，而深度学习 API 需要具有明确类型和结构的张量及属性参数，因此需要相应测试驱动完成输入转换。FlashFuzz 利用大语言模型结合驱动模板、领域辅助函数及 API 文档自动生成测试驱动，并通过反馈不断修复生成结果，使覆盖引导模糊测试能够更有效地应用于 TensorFlow、PyTorch 等深度学习框架 API。

总体而言，API 初始输入生成已经从已有调用样本和人工规则逐渐发展到文档约束提取、源码分析、运行反馈学习、路径约束求解以及大语言模型辅助生成等多种方法。研究重点也由最初的“生成能够成功执行的输入”逐步转向“生成能够触达更多程序行为和深层执行路径的输入”。

(2) API 输入突变

在 API 级模糊测试中，输入突变是一种常见且高效的手段。其核心思想是从已有的种子输入出发，通过对输入参数进行系统化修改，衍生出数量更多、分布更广的测试数据。这种方式无需依赖复杂的约束建模，就能快速生成合法性较高且贴近实际应用场景的输入，从而降低测试策略设计与实现的难度。输入突变的典型过程包括种子输入的选择、突变规则的设计与应用、测试执行、种子库的动态更新，以及输出行为的监测与分析等环节。其中，突变规则的合理性直接决定了突变结果的多样性与缺陷触发能力，因此通常需要结合深度学习框架 API 的特性进行针对性设计。通过这种方式，输入突变不仅能够提升测试效率，还能有效扩展探索范围，进一步增加发现潜在缺陷的可能性。

在输入生成与突变方面，FreeFuzz 设计了一套较为系统的规则，用于扩展 API 的测试输入空间。其方法主要包括类型突变和数值突变两类：前者通过改变张量维度、数据类型，或替换基本数据类型、元组与列表元素来提升输入结构的多样性；后者既可以通过随机方式生成新的张量形状、张量取值或基本类型参数，也可以从已有参数数据库中抽取典型值作为输入，以保证突变后的参数在合法性和代表性之间取得平衡。通过这种组合策略，FreeFuzz 能够高效生成大量具有差异性的输入，从而提高测试覆盖率并发现计算错误与性能缺陷。

IvySyn^[46]在传统基于变异的模糊测试基础上引入了类型感知变异，针对深度学习框架常见的复杂数据结构（如张量和列表）进行定向测试。与仅依赖字节级变异的方法不同，IvySyn 能够生成通过参数类型检查的输入，从而有效触发更

深层次的执行路径。研究者在总结约 240 个 TensorFlow CVE 与涉及 PyTorch 内存安全的缺陷报告后,设计了一系列专门的变异策略:包括生成含有极端数值、空形状或多维度的张量,构造包含边界值或超长字符串的基本类型输入,以及基于原始参数的排列组合来扩展测试空间。通过构建类型相关的变异池并系统化遍历, IvySyn 能够在黑箱测试模式下快速探索潜在缺陷点。

TitanFuzz^[41] 是将大语言模型 (LLM) 引入深度学习库模糊测试的代表性工作,其主要创新在于提出了一种基于填充式大语言模型 (infilling LLM, 如 Incoder^[47]) 的输入变异机制。与传统依赖手工规则或随机扰动的模糊测试不同, TitanFuzz 不再局限于预设的简单突变规则,而是通过在种子代码的关键位置插入占位符 (), 利用 LLM 自动补全生成新代码。具体而言, TitanFuzz 通过参数替换、关键字插入、前缀/后缀插入和方法替换等变异操作,灵活改变 API 调用方式,生成额外的输入准备或后续处理代码,并通过替换 API 本身触发更广泛的行为差异。通过这些操作, TitanFuzz 能够系统性地扩展输入空间,生成更多样且具有代表性的测试用例。这一机制充分展示了大语言模型在驱动代码变异和输入生成方面的潜力。

DFuzz^[48] 同样属于大模型驱动输入突变方法,其核心是利用边界情况引导已有测试程序的定向变异。该方法首先利用 LLM 从深度学习库底层源码中的检查语句中抽取上下文无关的边界情况,并结合目标 API 的参数类型模式 (etype pattern) 筛选出可迁移和适用的边界条件。在变异阶段, DFuzz 将这些边界情况与初始程序结合,借助 LLM 生成可能触发异常的测试代码,从而提升输入的针对性与缺陷触发能力。为兼顾效率与覆盖率, DFuzz 将边界情况划分为单变量和复合两类:前者对关键参数进行重点覆盖,后者则对非关键参数按比例采样,在降低开销的同时保证测试的深度和代表性。

与其他方式不同, Orion^[49] 通过对大量历史漏洞 (如 CVE、补丁提交和安全报告) 进行根因分析,总结出输入模式与缺陷触发之间的对应关系,并将其提炼为一系列启发式突变规则。这些规则不仅覆盖了张量维度不匹配、索引越界、类型转换错误等常见问题,还包括 corner-case 输入的构造方式。在实际测试过程中, Orion 并非直接生成输入,而是以文档、开发者测试和开源仓库中收集的种子输入为基础,利用规则对其进行定向突变,从而高效探索潜在缺陷。相比 FreeFuzz 的通用突变策略或 DeepREL 的等价规则差分方法, Orion 更强调将突变操作与漏洞根因分析紧密结合,在保持输入合法性的同时显著提升触发真实缺陷的能力。

总体来看, API 输入突变已经从以类型、形状和数值为主的通用规则变异,逐渐发展到结合代码语义、边界条件和历史缺陷信息的定向突变。输入突变所利用的信息不断丰富,但如何进一步刻画 API 参数之间的细粒度依赖关系,并根

据运行状态动态决定突变哪些参数以及如何协调相关字段变化，仍有进一步研究空间。

综上所述，现有面向深度学习框架的 API 级模糊测试在初始输入生成、约束提取、路径探索和输入突变等方面已经取得了一定进展，但仍存在以下不足：

首先，在有限测试时间和计算资源约束下，如何合理分配不同 API 的测试资源仍然是一个重要问题。现有工具通常对所有 API 分配相同的测试时间或测试次数，但不同 API 在代码复杂度、最近修改时间和历史修改频率等方面存在差异，其潜在缺陷风险可能并不相同。因此，如果忽视 API 之间的差异，均匀分配测试资源就可能导致部分 API 被反复测试，而另一些更值得关注的 API 未得到充分探索。如何提取可自动获取、可量化的风险信号，并据此指导测试资源分配，是提升 API 级模糊测试效率的重要问题。

其次，复杂约束下有效输入难以获取，分支覆盖难以持续扩展。张量形状、类型和属性之间存在复杂约束，已有输入生成过程可能难以充分刻画这些要求，导致输入反复被前置检查拒绝。即使获得有效输入，也可能持续触达已有分支，出现覆盖增长停滞。因此，如何结合错误信息、已有近似路径条件与执行样本，提取有助于后续探索的约束线索，并通过执行检验其对有效输入获取与新分支探索的作用，是需要解决的问题。

最后，细粒度参数关系复杂，输入突变的有效性与针对性仍需提高。高层 API 输入在测试驱动中往往被展开为细粒度 harness 参数，这些字段之间存在生效依赖、结构依赖与取值约束。例如，rank 决定哪些维度字段生效，axis 的取值范围通常依赖张量 rank。忽略这些关系容易修改当前未生效的字段或破坏输入约束。此外，历史缺陷报告包含与失败相关的参数和值，仍需将这些线索映射到驱动字段，用于运行时参数选择与取值采样。如何感知上述关系并协调相关字段的调整，是提升突变探索能力的重要问题。

从发展趋势来看，面向深度学习框架的 API 级模糊测试正由通用化、粗粒度的输入探索，逐步向风险感知、约束驱动和参数关系感知的精细化测试方向发展。一方面，测试资源分配开始更加关注 API 之间的差异，以提高有限测试资源的利用效率；另一方面，输入生成与突变逐渐融合运行反馈、路径约束、参数关系和历史缺陷等信息，以增强复杂输入空间与深层执行行为的探索能力。同时，符号执行、覆盖反馈和大语言模型等技术的结合，也为提升测试自动化程度和缺陷触发能力提供了新的途径。

二、课题主要研究内容、预期目标

2.1 研究内容

针对深度学习框架 API 数量庞大、输入约束复杂、缺陷触发条件隐蔽等问题，本研究围绕 API 级模糊测试中的资源分配、输入生成和输入突变三个关键环节开展研究，目标是在有限测试时间和计算资源下提升测试用例的有效性、针对性和缺陷触发能力。具体研究内容如下：

第一，API 风险评估与测试资源分配研究。针对不同 API 潜在风险存在差异、有限预算难以合理分配的问题，从 API 实现代码的演化特征出发，提取最近修改时间和历史修改频度，计算相对风险评分，并按评分占比分配测试预算，使评分较高的 API 获得更多测试资源，同时设置单个 API 的预算上限。

第二，面向复杂约束的 API 输入生成增强研究。针对有效输入获取受阻和覆盖增长停滞，拟结合错误信息、选定近似路径条件、代表性执行样本、驱动源码及 API 参数说明，构造具有明确适用范围的候选辅助约束；通过约束检查和临时辅助求解生成输入，并以真实执行结果更新使用记录和探索样本，保留原有路径条件合成与求解机制。

第三，参数感知的 API 输入突变研究。针对细粒度 harness 参数间的生效依赖、结构依赖与取值约束，研究活跃参数筛选、耦合结构突变和上下文感知取值方法，并将历史缺陷报告中的参数级和值级先验映射到驱动字段。在连续突变未产生新覆盖时启用缺陷引导，增强对易错参数组合与边界值的探索。

如图 2-1 所示，三项研究分别对应测试资源分配、输入生成和输入突变环节。风险评估为生成与突变过程提供预算分配依据；输入生成增强为后续突变拓展可用起点；参数感知突变进一步探索相关参数组合与边界条件。图中表示研究内容的功能关系，当前各项方法分别实现或设计，并独立评估。

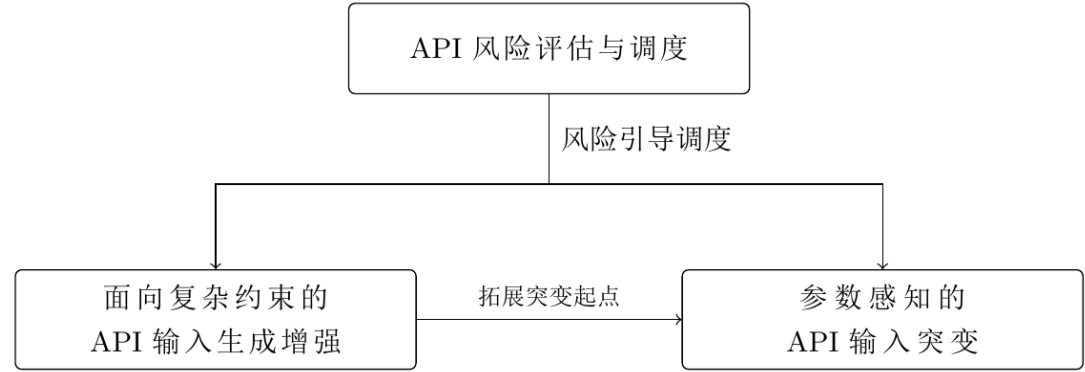


图 2-1 研究架构

2.2 预期目标

围绕上述研究内容，本研究拟实现以下预期目标：

1. 提出一种基于代码演化特征的 API 风险评估与测试资源分配方法，在有限测试预算下提高重点 API 的探索充分性和整体缺陷发现效率；
2. 提出一种面向复杂约束的 API 输入生成增强方法，提高深度学习框架 API 测试输入的有效性，并增强对深层执行路径的覆盖能力；
3. 提出一种参数感知的 API 输入突变方法，利用参数结构关系、运行时反馈和历史失败知识提升易错参数组合的探索能力。

总体而言，本研究围绕深度学习框架应用接口的模糊测试优化方法展开，旨在有限测试预算下提高有效输入获取能力与分支覆盖水平，增强深度学习库缺陷发现能力与测试效率。

三、学位论文进展情况、已取得阶段性成果

围绕测试资源分配、输入生成和参数突变三个环节，本研究分别开展代码演化特征驱动的 API 风险评估与测试资源分配、复杂约束下的输入生成增强和参数感知突变研究。

目前，研究内容一已完成 API 风险评估与测试资源分配方法的设计、实现和实验评估，相关论文发表于 ICASSP 2026；研究内容三已完成方法实现和实验评估，相关论文已投稿；研究内容二已形成初步技术方案，仍需完成代码实现与系统实验验证。

3.1 基于代码演化特征的 API 风险评估与测试资源分配方法

3.1.1 研究背景

不同 API 的潜在缺陷风险存在差异，均匀分配测试预算可能使高风险 API 得不到充分探索。回归灰盒模糊测试研究表明，联合利用最近修改时间和历史修改频率引导测试资源分配，有助于提高缺陷发现效率^[50]。本研究在此基础上，将代码演化信息用于深度学习框架 API 风险评估，计算不同 API 的相对风险评分，并据此分配有限测试预算。

3.1.2 研究内容

(1) API 风险评估

首先，通过静态分析定位 API 对应的底层实现函数及相关代码，明确演化特征的统计范围；随后，从代码修改记录中提取各代码行距最近一次修改的时间

间隔，以及相关代码的历史修改活跃程度。对于第 i 行代码，设 d_i 为其最近一次修改与评估时间点之间的时间间隔，使用指数衰减函数计算权重：

$$w_i = \exp\left(-\frac{d_i}{\lambda}\right). \quad (1)$$

其中， λ 为时间衰减尺度。修改越近， w_i 越接近 1；时间间隔增大时，权重逐渐降低，使近期代码变更在评分中获得较高权重。

设 N 为 API 相关实现代码的行数， U 为表征历史修改活动的统计量。沿用已有实现， U 按相关修改记录中的不同修改日期数量统计，用于刻画历史修改频度。API 风险评分为：

$$S_{api} = \frac{1}{N} \sum_{i=1}^N w_i \cdot (\log(U + c))^\beta. \quad (2)$$

该公式将各代码行的时间衰减权重与历史修改因子结合，再求和并除以代码行数 N 。对数变换缓和修改活动数量差异的影响， β 调节历史修改因子的作用强度，按代码行数取平均则减小代码规模差异对评分的直接影响。参数 λ 、 c 和 β 在历史缺陷数据上以 ROC-AUC 为指标进行选择。评分刻画 API 之间的相对风险，并用于预算分配，不直接表示经校准的缺陷发生概率。

(2) 风险引导的测试预算分配

得到 API 风险评分后，按评分占比分配剩余测试预算。设 R 为当前待分配预算的 API 集合， B_{remain} 为剩余测试预算， B_{max} 为单个 API 的预算上限，则本次分配为：

$$B_{api} = \min\left(\frac{S_{api}}{\sum_{j \in R} S_j} \cdot B_{remain}, B_{max}\right). \quad (3)$$

当按比例计算的预算超过 B_{max} 时，采用预算上限；否则采用计算结果。该策略使评分较高的 API 获得更多测试资源，同时限制资源过度集中。时间分配策略可以接入不同 API 级模糊测试工具，其收益通过同一工具接入策略前后的对比进行评估。

基于上述风险评估与测试资源分配方法，实现了 API 级模糊测试工具 RiskFuzz，整体结构如图 3-1 所示。工具将 API 风险评分用于测试资源分配，结合测试用例生成与执行反馈开展缺陷检测。

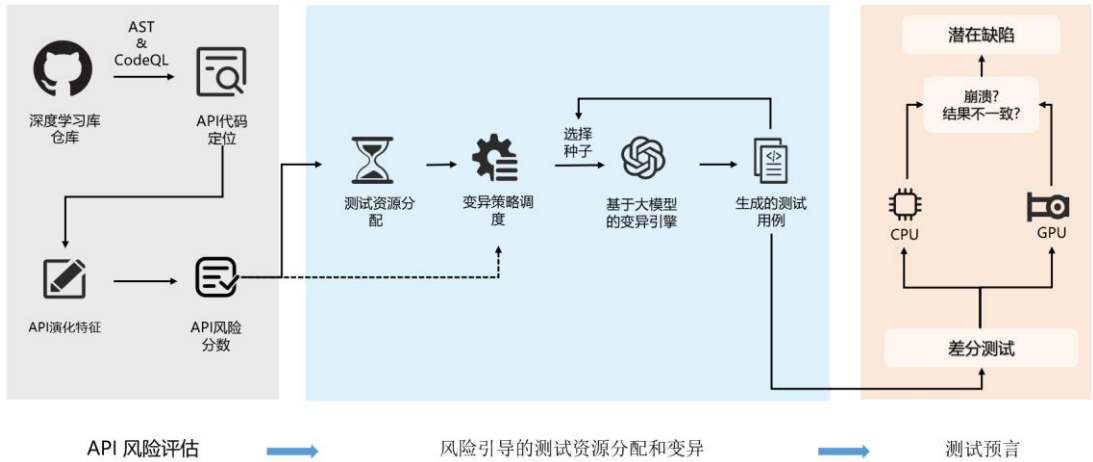


图 3-1 RiskFuzz 总体结构

3.1.3 实验评估

实验围绕风险评估与预算分配两个方面展开：通过统计已知缺陷 API 在风险排序中的分布，考察评分的风险区分能力；在相同测试预算下，对比已有模糊测试工具接入风险引导时间分配策略前后的缺陷发现数量，评估该策略的收益。另将 RiskFuzz 与基线工具进行对比，补充展示工具的整体缺陷发现效果。

实验结果表明，风险评分在排名前 30% 和前 50% 区间包含的已知缺陷 API 比例高于随机期望，具有一定的风险区分能力。在相同测试预算下，将风险引导时间分配策略接入已有工具后，缺陷发现数量均有所增加，表明依据 API 风险差异分配测试预算能够在所评估的工具上带来缺陷发现收益。

此外，RiskFuzz 在 TensorFlow 和 PyTorch 上均比所比较的基线工具发现了更多缺陷，且发现了 3 个此前未知缺陷。

3.1.4 已取得的阶段性成果

论文：

- Pan Z, Chen X, Zhang Z, et al. RiskFuzz: Risk-Guided Fuzzing for Deep Learning Libraries[C]//ICASSP 2026-2026 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). IEEE, 2026: 13837-13841. (已发表, CCF-B, 第一作者)

3.2 面向复杂约束的 API 输入生成增强方法

3.2.1 研究背景

以 PathFinder^[44] 为代表的方法根据执行样本合成近似路径条件，并通过约束求解生成 API 输入。在复杂输入约束下，部分 API 的有效输入获取仍可能受阻；即使已有有效输入，后续生成也可能反复触达已有分支，出现覆盖增长停滞。错误信息能够揭示部分输入要求，近似路径条件与执行样本能够反映已有探索情况，因而可以作为辅助输入生成的线索。

例如，在输入张量为三维的 TensorFlow Conv2D 测试中，可以观察到报错 Shape must be rank 4 but is rank 3。结合固定测试驱动中字段与张量构造的对应关系，可将该信息转化为候选要求 $x_rank == 4$ 。这一要求仅说明该次失败暴露出的维数条件，满足它并不保证其他形状、类型或属性约束也得到满足，仍需通过真实执行检验。

本研究拟解决两个问题：第一，在有效输入获取受阻或覆盖增长停滞时，如何结合错误信息、选定路径条件与代表性样本，以及驱动源码和 API 参数说明，提出具有明确适用范围的候选辅助约束；第二，如何检查候选约束与已有条件的兼容性，将其用于临时辅助求解，并根据执行结果更新使用记录，在保留常规探索的同时改善有效输入获取与覆盖扩展。

3.2.2 研究方法

本研究保留现有输入生成工具的近似路径条件合成、约束求解和执行反馈流程，增加面向测试状态的辅助约束生成机制。大模型用于辅助构造候选约束，具体输入由求解器生成，并由固定测试驱动执行验证。整体流程如图 3-2 所示。

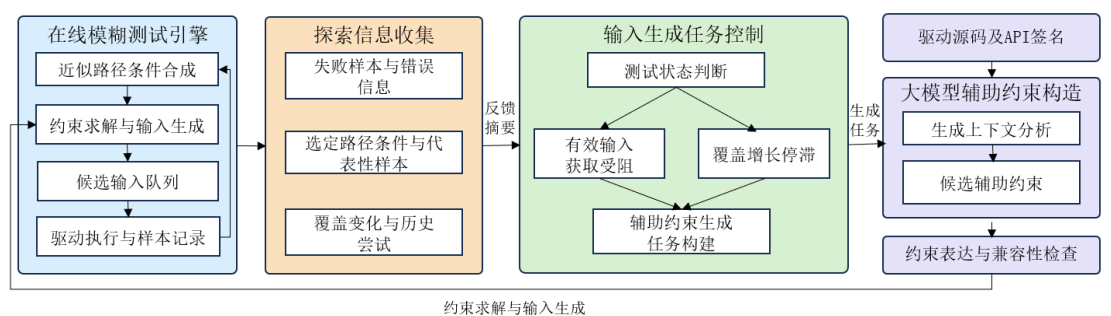


图 3-2 面向复杂约束的 API 输入生成增强框架

(1) 在线模糊测试引擎

保留原有的常规输入生成流程：根据执行样本合成近似路径条件，通过约束求解生成候选输入，并由固定测试驱动执行。执行产生的样本继续参与后续路径条件合成，形成持续探索的循环。辅助机制不替代这一流程，而是在需要时提供候选辅助约束，由引擎完成相应的求解、输入生成与执行。

（2）探索信息收集

随测试执行持续收集三类信息：一是失败样本及其错误信息，用于保留输入被拒绝的具体表现；二是已有近似路径条件及对应的代表性执行样本，用于描述已探索的输入配置；三是覆盖变化及历史辅助尝试的结果，用于反映探索进展。收集时保留输入、调用状态与执行结果之间的对应关系，并对重复错误和相似样本进行归并，形成供后续状态判断与任务构建使用的反馈摘要。本模块负责信息积累与整理，不决定是否启动辅助约束生成。

（3）输入生成任务控制

根据近期有效执行情况和覆盖变化判断测试状态。当连续生成的输入未能通过目标 API 的输入检查时，判定有效输入获取受阻，构建以补充输入要求为目标的辅助任务；当已有有效输入但连续探索未获得新分支时，判定覆盖增长停滞，构建以尝试不同输入配置为目标的辅助任务。未满足触发条件时，继续常规探索。

任务构建阶段再从已收集的信息中选择与当前问题相关的材料：对于有效输入获取受阻，选取失败样本、对应报错及相关路径条件；对于覆盖增长停滞，选取待继续探索的路径条件、代表性样本及历史尝试记录。任务明确本轮生成目标、涉及的输入字段和参考条件，为后续候选约束构造提供范围限定。状态判断的统计窗口与触发阈值将在实现和实验中确定。

（4）大模型辅助约束构造

将辅助任务与驱动源码、API 签名等上下文共同提供给大模型。其中，驱动源码用于说明输入字段如何构造成 API 实参，API 签名用于明确参数名称与调用形式；必要时补充相关参数说明。在有效输入获取受阻的情况下，结合报错与驱动字段映射，将错误信息揭示的输入要求转化为候选辅助约束，例如将维数要求映射为驱动中的秩字段约束。

在覆盖增长停滞的情况下，结合选定路径条件、代表性样本及历史尝试，提出用于本轮输入生成的候选探索约束，引导求解器尝试不同的参数配置。模型输出需明确约束涉及的字段、条件关系及适用范围，而不是直接输出测试输入。此类约束仅是待验证的探索建议，不能预先认定其对应未覆盖路径，也不能将其视为 API 的普遍合法性要求。

（5）约束表达与兼容性检查

检查候选约束的语法格式、字段名称、字段类型及适用范围，确认其能够转换为求解器可处理的形式，再检查其与选定路径条件是否能够同时满足。设 P 为选定的原有近似路径条件， C 为候选辅助约束， D 为固定驱动的基本表示约束，则由引擎在临时查询中尝试求解：

$$D \wedge P \wedge C. (4)$$

若联合条件可满足，则将求解得到的参数赋值送入原有候选输入队列；若确认存在冲突，则撤回本次候选辅助约束，并保留原有探索流程。求解超时或返回未知结果时，记录尝试状态，不将其直接判定为约束冲突。辅助约束仅用于相应临时查询，不直接修改或永久追加到原有路径条件中。

候选输入回到在线模糊测试引擎后，由固定驱动执行，检查其是否通过目标调用、是否增加分支覆盖及是否触发异常或崩溃。执行样本按原有机制参与后续近似路径条件合成，辅助约束的使用情况与执行结果则纳入探索信息收集，为下一轮状态判断与任务构建提供反馈。求解成功并不保证输入有效，一次有效执行也不意味着候选约束普遍成立。

目前，该研究内容已形成初步技术方案，代码实现与系统实验尚待完成。后续将完成辅助约束生成与现有输入生成流程的集成，并在相同测试预算下，从有效输入获取、分支覆盖、缺陷发现及额外开销等方面评估方法效果。

3.3 参数感知的 API 输入突变方法

3.3.1 研究背景

当深度学习框架 API 的测试输入被 API 测试驱动表示为一组细粒度的 harness 参数时，一个高层 API 输入可能会被拆分为多个可生成和可突变的底层字段，例如张量的数据类型、秩、维度和元素值，列表或数组的长度与元素，以及布尔或枚举属性等。在这种表示下，这些 harness 参数并不是相互独立的：部分参数决定其他参数是否参与实际 API 调用构造，部分参数则与其他参数的取值解释提供结构上下文。若突变过程忽视这些关系，就可能反复修改对当前 API 调用没有影响的非活跃参数，或在修改结构控制参数时未同步调整相关参数，从而降低覆盖探索效率和输入有效性。针对以上问题，本研究提出了参数感知的 API 输入突变方法 ParaFuzz，其整体流程如图 3-3 所示

3.3.2 研究内容

ParaFuzz 的研究出发点是：现有输入生成方法能够帮助模糊器获得可执行种子，但后续测试效果很大程度上取决于这些种子如何被继续突变。当测试输入被表示为细粒度 harness 参数时，突变器需要回答三个问题：哪些参数在当前种子下真正参与 API 输入构造，哪些参数之间需要联动更新，以及哪些参数和值更可能与历史失败相关。为此，ParaFuzz 在参数层面进行突变，并将参数结构关系、历史缺陷报告中的失败知识和运行时覆盖反馈转化为可执行的突变决策。

在参数表示方面，ParaFuzz 利用 API 测试驱动显式暴露出的参数化表示来构造和维护有效种子。以张量输入为例，一个高层 API 参数通常会被拆分为数据类型、rank、多个维度参数以及元素值；对于列表或数组参数，则可能被拆分

为 size 参数和若干 element 参数；对于布尔或枚举属性，则会被表示为可选择的符号参数。这种参数化表示为后续突变提供了更细粒度的操作对象，也使得模糊器能够分析不同参数之间的依赖关系。

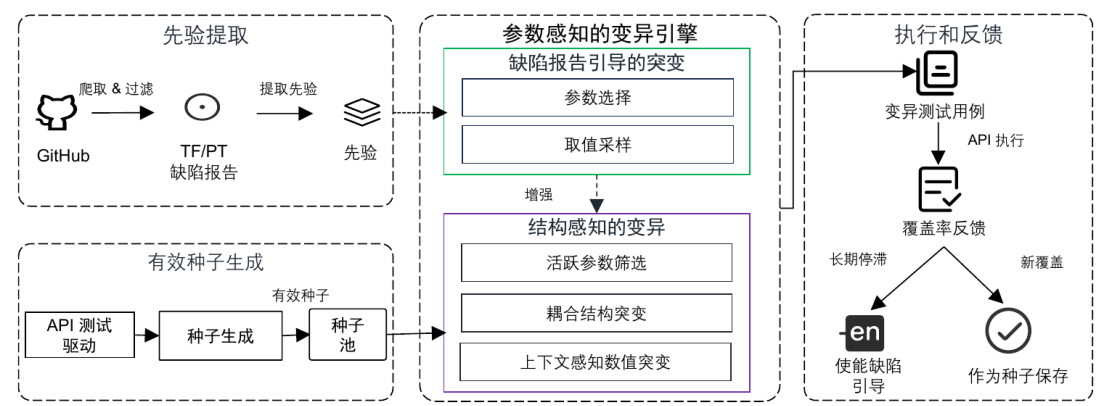


图 3-3 ParaFuzz 参数感知输入突变框架

ParaFuzz 包含两类互补的突变策略：结构感知突变和缺陷报告引导突变。结构感知突变主要服务于覆盖导向探索，目标是在已有有效种子附近更高效地扩展输入空间；缺陷报告引导突变主要服务于缺陷导向测试，目标是利用历史真实失败中出现过的参数和值，引导测试过程关注更可能触发崩溃或异常的参数上下文。两类策略通过运行时分支覆盖反馈进行协调：在普通探索阶段，系统优先执行结构感知突变；当连续多次突变未带来新的分支覆盖时，再启用缺陷报告引导突变，同时保留活跃参数筛选与结构参数处理机制，以增强对易错参数和值的定向探索。执行成功且带来新分支覆盖的输入被保留为种子，供后续突变使用。

（1）结构感知突变

ParaFuzz 首先识别测试驱动参数之间的结构关系，并据此进行突变目标选择、联动更新和取值采样。具体而言，rank-shape 关系表示 rank 参数决定哪些维度参数参与张量形状构造；size-element 关系表示 size 参数决定哪些元素参数参与数组或列表构造；axis、dim、batch_dims 等维度选择参数的合法范围通常依赖输入张量的秩；indices、ids、offsets 等索引类参数的取值往往受到输入张量形状约束；stride、dilation、padding、kernel size 等窗口或形状参数则会共同影响输入输出张量的形状关系。围绕这些关系，ParaFuzz 设计了三类轻量级突变机制。

第一，活跃参数筛选。对于同一个种子，并非所有 harness 参数都会影响最终 API 调用。例如，当某个张量的 rank 为 2 时，超过前两个维度槽位的维度参数不会参与实际张量构造；当某个数组长度为 1 时，超出长度范围的元素参数也不会被使用。ParaFuzz 在每次突变前根据当前种子的结构信息筛选真正生

效的参数，并优先从这些活跃参数中选择突变目标，从而减少对无效参数的重复修改。

第二，耦合结构突变。部分参数本身会改变其他参数是否生效或如何解释，例如 `rank`、`array size` 等结构控制参数。如果只修改这类参数，而不同时更新新激活的维度或元素参数，生成输入很容易被形状检查、长度检查或张量构造逻辑拒绝。ParaFuzz 因此在修改结构控制参数后，同步重新采样相关的维度或元素值，使新的输入结构能够立即参与后续测试。

第三，上下文感知取值突变。对于 `axis`、`index`、`stride`、`dilation`、`padding` 等参数，其合法范围和实际语义通常依赖于输入张量的秩、形状或布局。ParaFuzz 不再将这些参数视为孤立数值，而是根据当前种子的结构上下文调整取值范围。例如，轴参数会结合可观测的张量 `rank` 生成候选值，索引类参数会结合张量维度生成负索引、边界索引或越界索引，窗口类参数则结合输入形状生成更有意义的步长和膨胀率组合。这些取值规则基于可观察的结构上下文进行近似判断，并保留边界及越界取值探索，不保证每个候选输入均满足 API 的完整语义约束。

(2) 缺陷引导突变

ParaFuzz 从 TensorFlow 和 PyTorch 的历史缺陷报告中收集与运行时崩溃、异常终止、断言失败、越界访问和内存错误等相关的信息，并从中抽取 API 名称、参数名称、张量形状、数据类型、特殊取值和错误上下文等内容。与简单重放历史缺陷输入不同，ParaFuzz 将这些信息转化为参数级和值级先验：参数级先验用于指导运行时的突变目标选择，使突变器更关注 `axis`、`indices`、`padding`、`strides`、`dtype`、`shape`、`dilation`、`groups` 等历史失败中频繁出现的参数或参数族；值级先验用于指导取值采样，使突变器更倾向于尝试 0、-1、极大整数、NaN、Inf、空张量、零维度、负维度、越界索引、不常见数据类型和特殊布局选项等历史失败相关值。

由于缺陷报告中的参数通常以用户可见 API 的形式出现，而测试驱动程序中的参数可能被拆分为更底层的 `rank`、`dim`、`size`、`element` 或 `enum` 参数，ParaFuzz 进一步建立缺陷报告参数与 harness 参数之间的映射关系。例如，缺陷报告中的 `dtype` 可映射到驱动程序中的数据类型枚举参数，`shape` 可映射到 `rank` 和维度参数，`indices` 可映射到索引张量或索引数组参数。若某个缺陷报告中的参数无法精确映射到当前驱动程序参数，ParaFuzz 不会强行将其作为直接突变目标，而是保留其中可复用的值模式作为通用先验。通过这一过程，历史缺陷报告中的失败知识被转化为运行时可用的参数选择和取值采样指导。

3.3.3 实验评估

实验结果表明，ParaFuzz 在 TensorFlow 和 PyTorch 上相较所评估的基线工具覆盖了更多分支，相较 PathFinder 触发崩溃的 API 数量也有所增加。消融实验显示，结构感知突变有助于改善覆盖探索和输入有效性，历史缺陷引导进一步增强了缺陷触发能力。测试中共提交 13 个缺陷报告，除 2 个重复报告外，其余 11 个此前未知缺陷均已确认并修复。

3.3.4 已取得的阶段性成果

论文：

- ParaFuzz: Parameter-Aware Fuzzing for Deep Learning Libraries（相关论文已投稿）

四、下一步工作计划和内容、预计答辩时间及存在的问题

下一步工作主要分为两个部分：首先完成研究内容二的方法设计、代码实现、实验评估及学术论文撰写；其次开展博士学位论文撰写，系统梳理博士期间的研究成果，完成论文修改与答辩准备。研究内容一已发表，研究内容三已完成论文投稿，后续整理相关代码、实验数据和缺陷案例，以支撑学位论文。

4.1 下一步工作计划

2026 年 9 月至 2026 年 12 月 完成面向复杂约束的 API 输入生成增强方法的技术方案、代码实现与实验评估，重点实现执行信息整理、辅助约束构造、兼容性检查、临时辅助求解及执行验证机制，并完成对应学术论文撰写。

2027 年 1 月至 2027 年 3 月 完成博士学位论文撰写，交由导师审阅，根据反馈修改完善；同时准备博士答辩 PPT，为预答辩做好准备。

2027 年 4 月至 2027 年 5 月 根据各位专家对博士学位论文的评审意见，进一步修改论文，完善研究内容、实验分析与论述质量。

2027 年 5 月至 2027 年 6 月 完成正式答辩准备，预计进行博士学位论文答辩。

4.2 预计答辩时间

预计于 2027 年 5 月至 2027 年 6 月进行博士学位论文正式答辩，具体时间根据论文评审进度及学校、研究所的安排确定。

4.3 存在的问题与挑战

整体研究方向保持不变，对研究内容二、三的技术路线进行了调整：研究内容二由‘特定参数组合与动态符号执行结合’调整为‘面向复杂约束的 API 输入生成增强’，利用错误信息、已有路径条件及执行样本辅助构造约束，改善有效输入获取与覆盖扩展；研究内容三由‘结合大模型生成的突变’调整为‘参数感知的 API 输入突变’，利用细粒度参数关系与历史缺陷先验指导参数选择、协调更新和取值采样。”

目前，研究内容二仍需进一步完善技术方案，完成代码实现与系统实验验证。

五、已取得研究成果列表（已发表、待发表学术论文等）

5.1 学术论文

1. Pan Z, Chen X, Zhang Z, et al. RiskFuzz: Risk-Guided Fuzzing for Deep Learning Libraries[C]//ICASSP 2026-2026 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). IEEE, 2026: 13837-13841. （已发表，CCF-B，第一作者）
2. Pan Z, Chen X, Chen L, et al. C-Fuzz: Fuzzing with Forced Program Control Flow Jumps[C]//2024 IEEE International Symposium on Parallel and Distributed Processing with Applications (ISPA). IEEE, 2024: 800-807. （已发表，CCF-C，第一作者）
3. ParaFuzz: Parameter-Aware Fuzzing for Deep Learning Libraries（相关论文已投稿）

5.2 科研项目参与

- 基于控制流完整性研究和数据流完整性的内存破坏攻击防御（国家自然科学基金，2021 年 3 月起）。
- 无人系统安全研究。

5.3 缺陷发现与修复

截至本次中期汇报，累计发现深度学习库缺陷 14 个，其中 12 个已修复。

按研究工作分别统计，RiskFuzz 发现 3 个此前未知缺陷；ParaFuzz 提交 13 个

缺陷报告，其中 2 个为重复报告，其余 11 个均已确认并修复。重复报告不计入累计新缺陷数量。

参考文献

- [1] Hinton G E, Salakhutdinov R R. Reducing the dimensionality of data with neural networks[J]. science, 2006, 313(5786): 504-507.
- [2] Deng L. Artificial intelligence in the rising wave of deep learning: The historical path and future outlook [perspectives][J]. IEEE Signal Processing Magazine, 2018, 35(1): 180-177.
- [3] Girija S S. Tensorflow: Large-scale machine learning on heterogeneous distributed systems[J]. Software available from tensorflow. org, 2016, 39(9).
- [4] Paszke A, Gross S, Chintala S, et al. Automatic differentiation in pytorch[J]. 2017.
- [5] Chen T, Li M, Li Y, et al. Mxnet: A flexible and efficient machine learning library for heterogeneous distributed systems[J]. arXiv preprint arXiv:1512.01274, 2015.
- [6] Seide F, Agarwal A. CNTK: Microsoft's open-source deep-learning toolkit[C]//Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining. 2016: 2135-2135.
- [7] Shen D, Wu G, Suk H I. Deep learning in medical image analysis[J]. Annual review of biomedical engineering, 2017, 19: 221-248.
- [8] Benedetti P, Femminella M, Reali G. Mixed-sized biomedical image segmentation based on U-Net architectures[J]. Applied Sciences, 2022, 13(1): 329.
- [9] Brown D. Hospitals turn to artificial intelligence to help with an age-old problem: Doctors' poor bedside manners. The Washington Post (2021)[EB/OL].(2021)
- [10] Zhang J, Qin Q, Ye Q, et al. ST-unet: Swin transformer boosted U-net with cross-layer feature enhancement for medical image segmentation[J]. Computers in Biology and Medicine, 2023, 153: 106516.
- [11] Bojarski M, Del Testa D, Dworakowski D, et al. End to end learning for self-driving cars[J]. arXiv preprint arXiv:1604.07316, 2016.

- [12] Grigorescu S, Trasnea B, Cocias T, et al. A survey of deep learning techniques for autonomous driving[J]. Journal of field robotics, 2020, 37(3): 362-386.
- [13] Zhang M, Zhang Y, Zhang L, et al. Deeproad: Gan-based metamorphic testing and input validation framework for autonomous driving systems[C]//Proceedings of the 33rd ACM/IEEE international conference on automated software engineering. 2018: 132-142.
- [14] Huval B, Wang T, Tandon S, et al. An empirical evaluation of deep learning on highway driving[J]. arXiv preprint arXiv:1504.01716, 2015.
- [15] TUGEND A. A smarter app is watching your wallet[N/OL]. The New York Times, 2021-03-09.
- [16] Leo M, Sharma S, Maddulety K. Machine learning in banking risk management: A literature review[J]. Risks, 2019, 7(1): 29.
- [17] Dixon M F, Halperin I, Bilokon P. Machine learning in finance: From theory to practice[M]. Berlin: Springer, 2020.
- [18] Fang Z, Ma X, Pan H, et al. Movement forecasting of financial time series based on adaptive LSTM-BN network[J]. Expert Systems with Applications, 2023, 213: 119207.
- [19] Ren X, Zhou P, Meng X, et al. Pangu- Σ : Towards trillion parameter language model with sparse heterogeneous computing[J]. arXiv preprint arXiv:2303.10845, 2023.
- [20] ZIEGLER C. A Google self-driving car caused a crash for the first time[N/OL]. The Verge, 2016-02-29.
- [21] CVE Details. CVE-2021-43811: Sockeye YAML vulnerability[EB/OL]. 2021-12-08.
- [22] Ji J, Kong W, Tian J, et al. Survey on fuzzing techniques in deep learning libraries[C]//2023 8th International Conference on Data Science in Cyberspace (DSC). IEEE, 2023: 461-467.
- [23] 马祥跃, 杜晓婷, 采青, 等. 深度学习框架测试研究综述[J]. 软件学报, 2024, 35(8): 3752-3784.
- [24] 张子涵, 赖清楠, 周昌令. 深度学习框架模糊测试研究综述[J]. 信息网络安全, 2024.

- [25] Pham H V, Lutellier T, Qi W, et al. CRADLE: cross-backend validation to detect and localize bugs in deep learning libraries[C]//2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE). IEEE, 2019: 1027-1038.
- [26] Guo Q, Xie X, Li Y, et al. Audee: Automated testing for deep learning frameworks[C]//Proceedings of the 35th IEEE/ACM international conference on automated software engineering. 2020: 486-498.
- [27] Wang Z, Yan M, Chen J, et al. Deep learning library testing via effective model generation[C]//Proceedings of the 28th ACM joint meeting on european software engineering conference and symposium on the foundations of software engineering. 2020: 788-799.
- [28] Zou Y, Sun H, Fang C, et al. Deep learning framework testing via hierarchical and heuristic model generation[J]. Journal of Systems and Software, 2023, 201: 111681.
- [29] Shen X, Zhang J, Wang X, et al. Deep learning framework fuzzing based on model mutation[C]//2021 IEEE Sixth International Conference on Data Science in Cyberspace (DSC). IEEE, 2021: 375-380.
- [30] Wu J, Li S, Li J, et al. DeepCov: coverage guided deep learning framework fuzzing[C]//2022 7th IEEE International Conference on Data Science in Cyberspace (DSC). IEEE, 2022: 399-404.
- [31] Li M, Cao J, Tian Y, et al. Comet: Coverage-guided model generation for deep learning library testing[J]. ACM Transactions on Software Engineering and Methodology, 2023, 32(5): 1-34.
- [32] Li J, Li S, Wu J, et al. Mmos: Multi-staged mutation operator scheduling for deep learning library testing[C]//GLOBECOM 2022-2022 IEEE Global Communications Conference. IEEE, 2022: 6103-6108.
- [33] Liu J, Peng J, Wang Y, et al. Neuri: Diversifying dnn generation via inductive rule inference[C]//Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 2023: 657-669.
- [34] Schumi R, Sun J. ExAIS: executable AI semantics[C]//Proceedings of the 44th International Conference on Software Engineering. 2022: 859-870.

- [35] Wei A, Deng Y, Yang C, et al. Free lunch for testing: Fuzzing deep-learning libraries from open source[C]//Proceedings of the 44th International Conference on Software Engineering. 2022: 995-1007.
- [36] Deng Y, Yang C, Wei A, et al. Fuzzing deep-learning libraries via automated relational api inference[C]//Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering. 2022: 44-56.
- [37] Yang C, Deng Y, Yao J, et al. Fuzzing automatic differentiation in deep-learning libraries[C]//2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). IEEE, 2023: 1174-1186.
- [38] Li Y. Documentation-guided fuzzing for testing deep learning api functions[J]. 2020.
- [39] Shi J, Xiao Y, Li Y, et al. Acetest: Automated constraint extraction for testing deep learning operators[C]//Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis. 2023: 690-702.
- [40] Kang H J, Rattanukul P, Haryono S A, et al. Skipfuzz: Active learning-based input selection for fuzzing deep learning libraries[J]. arXiv preprint arXiv:2212.04038, 2022.
- [41] Deng Y, Xia C S, Peng H, et al. Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models[C]//Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis. 2023: 423-435.
- [42] Deng Y, Xia C S, Yang C, et al. Large language models are edge-case fuzzers: Testing deep learning libraries via fuzzgpt[J]. arXiv preprint arXiv:2304.02014, 2023.
- [43] Li Z, Wu J, Ling X, et al. The seeds of the future sprout from history: Fuzzing for unveiling vulnerabilities in prospective deep-learning libraries[C]//2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE). IEEE, 2025: 1616-1627.
- [44] Kim S, Kim Y, Park D, et al. Lightweight Concolic Testing via Path-Condition Synthesis for Deep Learning Libraries[C]//2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE). IEEE, 2025: 2957-2969.

- [45] Qin F, Naziri M M, Ai H, et al. Evaluating the effectiveness of coverage-guided fuzzing for testing deep learning library APIs[J]. arXiv preprint arXiv:2509.14626, 2025.
- [46] Christou N, Jin D, Atlidakis V, et al. {IvySyn}: Automated vulnerability discovery in deep learning frameworks[C]//32nd USENIX Security Symposium (USENIX Security 23). 2023: 2383-2400.
- [47] Fried D, Aghajanyan A, Lin J, et al. Incoder: A generative model for code infilling and synthesis[J]. arXiv preprint arXiv:2204.05999, 2022.
- [48] Zhang K, Wang S, Han J, et al. Your fix is my exploit: Enabling comprehensive DL library API fuzzing with large language models[C]//2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE). IEEE, 2025: 3110-3122.
- [49] Shiri Harzevili N, Mohajer M M, Wei M, et al. History-driven fuzzing for deep learning libraries[J]. ACM Transactions on Software Engineering and Methodology, 2025, 34(1): 1-29.
- [50] Zhu X, Böhme M. Regression greybox fuzzing[C]//Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security. 2021: 2169-2182.